TACITVS

Protocol Security Audit Report

Sine ira et studio.
"WITHOUT ANGER OR BIAS."

Tacitus, Annals 1.1

TACITVS

Auctor Princeps

January 24, 2026

Contents

1 Table of Contents **2**

2 Protocol Summary **2**

3 Disclaimer **3**

4 Scope & Methodology **3**

4.1 Audit Scope 3

4.2 Methodology 3

4.3 Exclusions 3

5 Risk Classification **4**

6 Audit Details **4**

6.1 Commit Hashes 4

6.2 Focus Areas 4

7 Executive Summary **4**

7.1 Issues Found 5

7.2 Key Observations 5

8 Findings **5**

8.1 Medium 5

8.1.1 Description 5

8.1.2 Impact 6

8.1.3 Proof of Concept 6

8.1.4 Recommended Mitigation 6

8.1.5 Severity Justification 7

8.2 Low 7

8.2.1 Description 7

8.2.2 Impact 8

8.2.3 Proof of Concept 8

8.2.4 Recommended Mitigation 9

8.2.5 Severity Justification 9

9 Appendix: Investigated But Not Vulnerable **9**

Proof & Connect **10**

Prepared by: TacitvsXI

Independent Security Researcher: I.Leskov

I. Table of Contents

- Table of Contents
- Protocol Summary
- Disclaimer
- Scope & Methodology
  - Audit Scope
  - Methodology
  - Exclusions
- Risk Classification
- Audit Details
  - Commit Hashes
  - Focus Areas
- Executive Summary
  - Issues Found
  - Key Observations
- Findings
  - Medium
  - Low
- Appendix: Investigated But Not Vulnerable

II. Protocol Summary

Olas (formerly Autonolas) is a unified network for off-chain services like automation, oracles, and co-owned AI. The protocol enables autonomous AI agents to operate on blockchain infrastructure.

The audit encompasses three main repositories:

1. **autonolas-governance** - Token voting, cross-chain governance execution via bridges, security guards (GuardCM), and token burning mechanisms.
2. **autonolas-registries** - Registration and management of components, AI agents, and deployed services. Services are deployed as multisig wallets (Safe) with recovery mechanisms and activity-based staking rewards.
3. **autonolas-tokenomics** - Token inflation schedules, staking rewards distribution, protocol-owned liquidity management, cross-chain buyback-and-burn mechanisms, and L2 staking dispensers.

III. Disclaimer

TacitvsXI (I.Leskov) makes all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security audit is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the Solidity implementation of the contracts.

Important Note on Tool-Assisted Findings: This audit was conducted alongside Zellics V12 AI auditor. All findings identified by V12 are out of scope per competition rules. The findings in this report represent **novel vulnerabilities not detected by V12 or other automated tools**.

IV. Scope & Methodology

A. Audit Scope

The audit covered 40 Solidity contracts across three repositories:

autonolas-governance (14 contracts): Cross-chain bridges, GuardCM, token burning

autonolas-registries (8 contracts): Service management, staking, recovery modules

autonolas-tokenomics (18 contracts): Inflation, dispensers, liquidity management, oracles

Solidity Version: 0.8.30
EVM Version: Prague
Optimizer: Enabled (750 runs)

B. Methodology

The audit followed a systematic approach:

1. **Reconnaissance** - Understanding protocol architecture, tokenomics model, and cross-chain design
2. **Architecture Mapping** - Tracing fund flows, trust boundaries, and state dependencies
3. **Critical Path Audit** - Deep analysis of highest-risk contracts (Tokenomics, bridges, staking)
4. **Invariant Analysis** - Identifying and verifying protocol invariants
5. **Cross-Contract Analysis** - Testing multi-step attack chains across contract boundaries
6. **PoC Development** - Validating findings with working exploit code

C. Exclusions

The following are **explicitly out of scope** per competition rules:

- All V12 AI auditor findings (comprehensive list provided by sponsor)
- Vulnerabilities in previous audit reports
- Known issues documented in vulnerability PDFs
- Inherited code (OpenZeppelin, Solmate, bridge contracts)
- Code inspired by Curve DAO, Uniswap, Maple Finance, Safe
- User misconfiguration issues

V. Risk Classification

Severity is determined using the CodeHawks methodology combining Impact and Likelihood:

- **Critical/High**: High impact + High likelihood
- **Medium**: High impact + Medium likelihood, OR Medium impact + High likelihood
- **Low**: Low impact OR Low likelihood combinations

We follow the standard C4 severity matrix for final classification.

VI. Audit Details

A. Commit Hashes

```
autonolas-governance:  e7b6039
autonolas-registries:  d4f7d34
autonolas-tokenomics:  bbec5ac
```

B. Focus Areas

Per sponsor guidance, special attention was given to:

- Re-entrancy vectors
- Integer overflow/underflow (despite Solidity 0.8+)
- Access control issues
- Price oracle manipulation

VII. Executive Summary

This audit identified **2 novel vulnerabilities** not detected by V12 or other automated tools:

1. **Medium**: `redeem()` function in `DefaultTargetDispenserL2` bypasses emissions limit changes made by governance after a request is queued
2. **Low**: `updateInflationPerSecondAndFractions()` in `Tokenomics` discards accumulated `effectiveBond` instead of adjusting it

Both findings were validated with working Proof of Concept tests using Foundry.

The V12 AI auditor proved remarkably comprehensive, identifying most vulnerability patterns including critical assembly parsing issues in bridge contracts, oracle manipulation vectors, and integer handling concerns. The findings in this report represent edge cases in business logic that require deep protocol understanding to identify.

A. Issues Found

- **Critical**: 0
- **High**: 0
- **Medium**: 1
- **Low**: 1
- **Info**: 0
- **Total**: 2

B. Key Observations

1. **V12 Coverage**: The AI auditor was exceptionally thorough, covering assembly parsing, integer handling, oracle issues, and initialization concerns
2. **Code Quality**: The codebase shows evidence of extensive prior auditing with well-documented functions and defensive coding patterns
3. **Cross-Chain Complexity**: The L1L2 bridging architecture introduces timing windows that can affect governance decisions
4. **Accumulator Logic**: State variables that accumulate values over time (`effectiveBond`) require careful handling during parameter updates

VIII. Findings

A. Medium

[M-1] redeem() Bypasses Emissions Limit Changes After Governance Updates
The `DefaultTargetDispenserL2::redeem()` function deposits OLAS to staking targets without re-verifying the target's current emissions limit, allowing bypass of governance-imposed restrictions applied after queueing.

Contract: `DefaultTargetDispenserL2.sol`
Function: `redeem()`
Lines: L299-336

I. Description

When `_processData()` receives messages from L1, it validates staking targets against their emissions limits via `verifyInstanceAndGetEmissionsAmount()`. If the contract is paused or has insufficient balance, requests are queued with the amount validated at queue time.

However, when `redeem()` is later called, it **only checks if the hash exists** - it does NOT re-verify the emissions limit:

```
function redeem(address target, uint256 amount, bytes32 batchHash) external {
    // ...
    bytes32 queueHash = keccak256(abi.encode(target, amount, batchHash, block.
        chainid, address(this)));
    bool queued = queuedHashes[queueHash];
    if (!queued) {
        revert TargetAmountNotQueued(target, amount, batchHash);
```

```
    }

    // NO call to verifyInstanceAndGetEmissionsAmount() here!

    IToken(olas).approve(target, amount);
    IStaking(target).deposit(amount);  // Full amount deposited
}
```

II. Impact

Governance Bypass: DAO governance can reduce emissions limits for problematic staking instances via: - `changeVerifier(newStricterVerifier)` - `setInstanceStatus(instance, false)` - Updating verifiers `getEmissionsAmountLimit()`

However, any amounts queued before these changes will still be deposited at the original (higher) limit.

Attack Scenario: 1. Staking instance X has emissions limit = 100,000 OLAS 2. L1 dispenser sends 100,000 OLAS incentive for X 3. L2 dispenser is paused request queued (amount=100,000) 4. DAO discovers X is misbehaving, reduces limit to 10,000 OLAS 5. Anyone calls `redeem(X, 100000, batchHash)` 6. Full 100,000 OLAS deposited, bypassing the new 10,000 limit

III. Proof of Concept

```
function test_PoC_RedeemBypassesEmissionsLimit() public {
    // Setup: staking target has emissions limit of 100 OLAS
    factory.setInstance(address(stakingTarget), 100 ether);

    // Pause dispenser to force queueing
    dispenser.pause();

    // Process message with 80 OLAS (within current limit)
    olas.mint(address(dispenser), 80 ether);
    bytes memory data = abi.encode(targets, amounts, batchHash);
    dispenser.processData(data);

    // DAO reduces emissions limit to 50 OLAS
    factory.updateEmissionsLimit(address(stakingTarget), 50 ether);

    // Unpause and redeem
    dispenser.unpause();
    vm.prank(attacker);
    dispenser.redeem(address(stakingTarget), 80 ether, batchHash);

    // VULNERABILITY: 80 OLAS deposited despite 50 OLAS limit
    assertEq(stakingTarget.totalDeposited(), 80 ether);
    assertGt(stakingTarget.totalDeposited(), 50 ether); // Exceeds limit!
}
```

Run: `forge test --match-contract RedeemBypassTest -vv`

IV. Recommended Mitigation

Add re-verification in `redeem()`:

```
function redeem(address target, uint256 amount, bytes32 batchHash) external {
    // ... existing checks ...
```

```
    // ADD: Re-verify target emissions limit
    bytes memory verifyData = abi.encodeCall(
        IStakingFactory.verifyInstanceAndGetEmissionsAmount, target
    );
    (bool success, bytes memory returnData) = stakingFactory.call(verifyData);

    uint256 limitAmount;
    if (success && returnData.length == 32) {
        limitAmount = abi.decode(returnData, (uint256));
    }

    // If target invalid or over limit, withhold excess
    if (limitAmount == 0) {
        withheldAmount += amount;
        queuedHashes[queueHash] = false;
        emit AmountWithheld(target, amount);
        return;
    }

    uint256 depositAmount = amount;
    if (amount > limitAmount) {
        withheldAmount += (amount - limitAmount);
        depositAmount = limitAmount;
    }

    // ... proceed with deposit using depositAmount ...
}
```

V. Severity Justification

- **Impact**: Medium - Governance decisions bypassed, excess OLAS deposited
- **Likelihood**: Medium - Requires queuegovernance changeredeem timing
- **Severity**: **Medium**

B. Low

[L-1] updateInflationPerSecondAndFractions() Discards Accumulated effectiveBond
When the owner recalibrates inflation parameters via `updateInflationPerSecondAndFractions()`, accumulated bonding capacity from refunded programs and epoch delays is silently discarded.

Contract: `Tokenomics.sol`
Function: `updateInflationPerSecondAndFractions()`
Lines: L1380-1381

I. Description

The `effectiveBond` variable accumulates bonding leftovers from previous epochs, as documented at line 308:

```
// effectiveBond = sum(MaxBond(e)) - sum(BondingProgram) over all epochs:
//                 accumulates leftovers from previous epochs
```

During normal `checkpoint()` operations, `effectiveBond` correctly accumulates:

```
// checkpoint() - Lines 1278-1280
curMaxBond += effectiveBond;
effectiveBond = uint96(curMaxBond);
```

However, in `updateInflationPerSecondAndFractions()`, the accumulated value is replaced:

```
// Lines 1380-1381
// Comment says "Adjust" but code REPLACES:
// Adjust effective bond with a new maxBond value issued for the on-going epoch
effectiveBond = uint96(curMaxBond);  // Discards accumulated value!
```

The comment at L1380 says Adjust which implies preserving accumulated value, but the implementation does a full replacement.

II. Impact

Lost Bonding Capacity: After multiple epochs of accumulation, `effectiveBond` can be significantly higher than a single epochs `maxBond`. Calling this function discards all accumulated capacity:

- 5 epochs of accumulation: ~30 OLAS effective bond
- After update: ~4 OLAS (reset to single epochs maxBond)
- **Lost**: ~26 OLAS of bonding capacity

Reduced Protocol Revenue: Future bond programs cannot utilize the lost capacity.

III. Proof of Concept

```
function test_PoC_EffectiveBondReset() public {
    // Run 5 epochs to accumulate effectiveBond
    for (uint i = 0; i < 5; i++) {
        vm.warp(startTime + ((i + 1) * EPOCH_LEN) + 1);
        tokenomics.checkpoint();
    }

    uint256 accumulated = tokenomics.effectiveBond();
    // accumulated  30 OLAS

    // Owner updates inflation parameters
    tokenomics.updateInflationPerSecondAndFractions(40, 30, 20, 10);

    uint256 after = tokenomics.effectiveBond();
    // after  4 OLAS

    assertLt(after, accumulated); // Lost 26 OLAS!
}
```

Run: `forge test --match-contract EffectiveBondResetTest -vv`

IV. Recommended Mitigation

Preserve accumulated value while updating:

```
// Get old maxBond
uint256 oldMaxBond = maxBond;

// Calculate new maxBond
uint256 curMaxBond = (curEpochLen * curInflationPerSecond * maxBondFraction) /
    100;

// Update maxBond
maxBond = uint96(curMaxBond);

// FIXED: Adjust effectiveBond by the difference
if (curMaxBond >= oldMaxBond) {
    effectiveBond = uint96(effectiveBond + curMaxBond - oldMaxBond);
} else {
    uint256 reduction = oldMaxBond - curMaxBond;
    if (reduction > effectiveBond) {
        effectiveBond = 0;
    } else {
        effectiveBond = uint96(effectiveBond - reduction);
    }
}
```

V. Severity Justification

- **Impact**: Low - Affects bonding capacity, not user funds
- **Likelihood**: Low - Owner-only function, rare inflation curve changes
- **Severity**: **Low**

Note: The comment/code mismatch strongly suggests this is unintentional behavior.

IX. Appendix: Investigated But Not Vulnerable

The following areas were investigated but determined to be secure or already covered by V12:

Treasury.sol - ETH overflow on receive(): Has explicit overflow check (L128-129)

Dispenser.sol - totalReturnAmount overflow: Solidity 0.8+ automatic checks

ArbitrumDepositProcessorL1 - Token/message race: Queuing handles timing

DefaultTargetDispenserL2 - Cross-chain replay: batchHash includes chainId

DefaultTargetDispenserL2 - Withheld double-counting: processedHashes prevents

StakingBase._evict - serviceId=0 escape: ID 0 never exists (covered by V12)

WormholeTargetDispenserL2 - Token atomicity: Wormhole ensures atomic delivery

GnosisTargetDispenserL2 - messageSender spoofing: AMB reliably returns L1 sender

VoteWeighting - Week boundary manipulation: 10-day vote delay prevents

veOLAS - Lock time gaming: Slope decay is deterministic

Report generated by TacitvsXI Security - January 2026

Proof & Connect

This report is cryptographically anchored on-chain. Scan the left QR to verify the exact SHA-256 fingerprint and blockchain transaction. Scan the right QR to explore my portfolio or book a call.

Scan to verify on-chain proof

Scan for portfolio & booking